DataBase Exercise

General

Submission date: 5.1.25 | Submission mode: Single | Exercise weight: 20%

In this exercise, you will practice and learn how to work with DataBases (DBs) using the ORM concept and technology.

The server logic will be the same one you have already developed during the past exercises: Books.
No pure-logic enhancements are planned nor needed for this exercise, though you will need to change how the system works behind the scenes (e.g. using persistence in DB)

Depending on the technology you have used, you will need to learn and explore how to work with the different ORMs to connect to the relevant DBs. The DBs will be given to you in the form of docker containers that you (and later the automation) will run locally on your machine.

This exercise will output a Docker image that you will create and upload to your Docker-hub account (the same as in the Docker exercise last semester).
Note: this exercise has nothing to do with the cloud exercise and your output is a new, independent, docker image, not something that runs constantly on the cloud.

We will use the docker-compose tool to describe and create the containers involved in this exercise (more details later)

An automation tool will test your code by pulling the image, building a live container out of it, invoking some HTTP requests against the container, and then verifying the DBs that hold the relevant data.
Note: In this exercise, the automation will ignore any log lines and will not expect or test them.

The Exercise

This exercise logic relies heavily on the *server & docker exercise*. The same logic is preserved, and all you need to do is enable it via a docker container, as was discussed and demonstrated in class (review the docker exercise for all the details related to uploading your image to the docker hub).

The aim of this exercise is to store and fetch the Books from a DB.
As part of the exercise you are given two docker containers that hold live DB instances
You should run them locally and make your application work with them.

Here are the details of the two containers:

1. **Postgres:**
   This is a common SQL DB.
   You can activate it with the below command (also listed in the attached **run-postgres-container.bat**)
   docker run -d --name postgres-db -p 5432:5432 ***theshultz/kaplat-books-postgres-instance:latest***
   This instance will be available on **localhost:5432.**

   DO NOT CHANGE THE PORTS!
   The automation will work with a similar container and will assume this port (**5432**) as well.
   DO NOT USE OTHER POSTGRES IMAGE!
   This is a special image with some changes that the automation will assume.

   The container connection details are listed below
   - User name: **postgres**
   - Password: **docker**
   - DB name: **books**
   - Table name: **books**

   The DB will have exactly one table that stores all the relevant Books.
   Here are the table fields and structure:
   - **rawid**: Integer. Unique key
   - **title**: String
   - **author**: String
   - **year**: Integer
   - **price**: Integer
   - **genres**: String. (holds an array of strings of the genres, all in capital case only)

2. **Mongo:**
   This is a common NoSql DB.
   You can activate it with the below command (also listed in the attached **run-mongo-container.bat**)
   docker run -d --name mongo-db -p 27017:27017 ***theshultz/kaplat-books-mongo-instance:latest***
   This instance will be available on **localhost:27017.**

   DO NOT CHANGE THE PORTS !
   The automation will work with a similar container, and will assume this port (**27017**) as well.
   DO NOT USE OTHER MONGO IMAGE !
   This is a special image with some changes that the automation will assume.

   The container connection details are listed below
   - Authentication: none
   - DB name: **books**
   - Collection name: **books**

   It stores documents (json's) that all have the same structure and attributes
   - **rawid**: Integer. Unique key
   - **title**: String
   - **author**: String
   - **year**: Integer
   - **price**: Integer
   - **genres**: String. (holds an array of strings of the genres, all in capital case only)

   Note: MongoDB uses a special, hard-coded field, called "**_id**" (that is: underscore id) throughout its documents. In order to be aligned with the table structure of Postgres, we are using a logical id field called **rawid**. Mongo will create an additional "**_id**" field for each stored document, which is transparent to you and you don't really need to fetch, expose or use it.

Keep in mind that the **rawid** field that serves as logical id in both Mongo and Postgres is something related to internal DB's issues.
The end client doesn't really care what you call the internal fields, and he still expects to get the field as was before, which is a simple "**id**", (e.g. when getting book details, using Postman)
In general: the overall structure of the returned JSON to the end user does not change from ex 3 & 5..

Both containers are pre-populated with several similar book records. Your application should reflect this information.
That means that once you are up and connected to the DBs, a call to 'get total books count' should return the number of already-stored books that the DB comes with.

Recall that each Book is assigned with a unique ID which is an ongoing integer.
As the DBs will already hold books, you will need to update your initial book counter to align with the data already in the DBs. (For example, if the DB holds two Books with IDs 1 and 2, then the next Book that will be created will be with ID 3.)
You are promised and can assume, and do not need to test or check, that the Books in both DBs will start from 1 (e.g., 1, 2, 3…) and will be aligned (that is, the first book on Postgres is the first book on Mongo, etc.).

The DB containers are configured to store and persist the data. When you stop the app container and start it again, the updates that were made to the DB will remain and be accessible after the restart.
In order to have a clean and fresh start, you should **delete** (not just stop) the db containers and then re-run them. This will create them from scratch based on the image with the initial state.

When you save\update Book data, it will be updated on both DBs.
When you fetch Book(s), you will be given an additional query parameter (persistenceMethod) that states from which DB you should fetch the data.
This query parameter can hold exactly one of these values (**case sensitive !**): **POSTGRES**, **MONGO**
This also enables you to double-check yourself: fetch the data from both of them, compare it, expect it to be similar, and return (one of) the (similar) results.

The below table states the various endpoints and how they behave with the DB implications

#	Endpoint Name	Method & resource	DB implications
1	Health	GET /books/health	nothing
2	Create new Book	POST /book	The new Book will be stored in both DBs. In case you encounter a logical error, it should not be stored at any of them.
3	Get (single) book details	GET /book?id Query parameter: persistenceMethod	Return a single book details from DB instance according to the supplied persistenceMethod. This does not change any data in the DBs
4	Get books count	GET /books/total Query parameter: persistenceMethod	Return the total count of books coming from DB instance according to the supplied persistenceMethod. This does not change any data in the DBs
5	Get books details	GET /books Query parameter: persistenceMethod	Returns the list of books details coming from DB instances according to the supplied persistenceMethod. This does not change any data in the DBs
6	Update Book price	PUT /book	Update the price of a book on both DBs. In case you encounter a logical error it should not be updated at any of them.
7	Delete Book	DELETE /book	Deletes the book instance from both DBs instances In case you encounter a logical error it should not be deleted from any of them.

During your work, both DBs should be aligned and reflect the same state of your application. As they run locally you are expected to be able to communicate with them easily without any errors and problems on your way.
Having said that, you might still encounter cases where, for some reason, you will be able to fetch data from one DB but not from the other. This is a real error and problem on your side that should not happen and basically shows that you can't connect with the DB.

In these cases you can choose to implement as you wish. You can return an error or you can return the information as was derived from only one DB or whatever and bet your chances.
When you are done developing the new server, you should prepare a new image of your application and push it to the docker hub.

The entire solution holds 3 containers (2 supplied DBs; 1 your app). It is best practice to describe these containers and the relationships between them using the **docker-compose** tool.
You can, and should, learn to work with docker-compose on your own. It's easy.
Here is an example (one of a gazillion out there…)

As part of this exercise, you are given a docker-compose.yml file to bootstrap the entire exercise setup. The docker-compose file already describes the two DB containers, and all you need to do is add a reference to the new docker image you built.

When working with the docker-compose setup you should consider the below:

- your app is assumed to work internally on port **8574** (as was in exercise 5) and it will be exposed externally on port **4785**.
- Your app container will share the same network with the DBs containers. As such:
  - You can access the Mongo DB using the name 'mongo' (e.g. **mongodb://mongo/books**)
  - You can access the Postgres DB using the name 'postgres' (e.g. **postgresql://postgres/books**
- Both DBs containers will be up and running before your app will be bootstrapped, so that when it comes up - the DBs are already available for connection.
  (in case you wondered: this is easily achieved using docker-compose ***depends_on*** capability)
- In order to start the whole scene simply invoke ***docker-compose up***
  (note**: *docker compose up -d*** will run the containers in the background)
- In order to stop the containers simply invoke ***docker-compose down***
  (note: **docker-compose stop** simply stops the containers. Botting them again will persist the DB status. Only the **down** command tears them apart so that next time they will start from scratch)

The automation will use the same docker-compose file you were given to bootstrap the scene. So, as usual with docker stuff, once you are able to "play it successfully" on your machine using the docker-compose, it will most likely also be played the same and as expected on the automation.

In addition, you are given a postman collection (it comes with the environment file you need to import as well that sets the port to 4785) through which you can test yourself:
The collection holds a series of requests, each one of which is a test phase in which it verifies the expected answer. Once you are up, you can execute it yourself (right-click -> run folder), and you should verify that you have passed all the tests.

Note: passing all tests in Postman almost surely promises your correct execution during the automation check.

What to Submit

You should submit a simple json file (**DO NOT** zip it !).
The json file must be in a valid json format. (**PLEASE, I BEG you** to use a json validator for that)

IT'S A MUST.
NO APPEAL WILL BE ACCEPTED FOR THIS REASON.
BY NOW, YOU ARE ALL BIG BOYS & GIRLS….

The json will hold the below data:

{
"**name**": string <your name>,
"**id**": string <your ID>,
"**imageName**": string "<full name of the image you created. That includes your repository prefix at docker hub along with the image tag name (after the ':' sign))>"
}

Example:

{
"name": "Menashe Hateymani",
"id": "98989898",
"imageName": "shul/server:1.0"
}

The automation will fetch the **imageName** attribute and will plant it in the correct place at the docker-compose file, similar to the one you used.
(Part of your feedback will hold the **docker-compose.yaml** file that the automation created and used as part of its test)

Note:

- The JSON file must be a valid json file. Note the use of (") wherever you need.
  In order to be sure it's a valid file, you can test yourself using an online json validator:
  https://jsonlint.com/
- The keys are **case-sensitive** and must appear **exactly** as they seem in the above example.
- Copying the json structure from this doc might yield the use of wrong characters due to different encoding schemes. For convenience, you can download and use this example file as a template for your file (**to make it clear**: you still need to check it is valid)